Data Science Capstone Project Report

SpaceX Falcon 9 First Stage Landing Prediction

Historical launch analysis and a reproducible classification benchmark

9 October 2026
Repository link requires completion before submission

Executive summary

The modeling snapshot contains 90 Falcon 9 launches: 60 Class 1 and 30 Class 0.

EDA and SQL describe differences across sites, orbits and years. Folium and Dash use a separate 56-record course snapshot.

SVM leads training CV accuracy at 88.86%. All four models score 77.78% on the 18-record test set.

The model detects all 12 Class 1 test records but misclassifies four Class 0 records. More data and chronological evaluation are needed.

Introduction and research problem

A successful first-stage landing can enable booster recovery and later reuse.

Project question: can historical launch features predict the course-defined first-stage landing outcome?

Compare LR, SVM, Decision Tree and KNN using the same data and evaluation protocol.

Explore launch-site geography and payload patterns to support descriptive planning insights.

The project estimates outcome classes. It does not estimate launch prices or financial savings.

Data sources and analysis scope

Source	Scope	Use
Archived API JSON	107 records	Launch and core fields
Modeling CSV	90 launches	EDA, SQL, classification
Geo / Dash CSVs	56 records each	Interactive exploration
Wikipedia HTML	2010-2019 page	Separate scraped tables

The two course snapshots have different scopes. Their success proportions must remain separate.

Data collection: SpaceX API

1. Download the archived API JSON from IBM and normalize the launch records.

2. Extract flight number, date, rocket ID and launchpad ID. Read landing_success from each core.

3. Read core landing_success separately from mission success. Align all 90 modeling rows to archived launches by unique date.

4. Use IBM enriched CSV as the fixed modeling reference. Live API calls returned HTTP 525 during preparation.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/01_Data_Collection_API.ipynb

API collection output

Flight	Mission	Date (UTC)	Mission success
1	FalconSat	2006-03-24	False
2	DemoSat	2007-03-21	False
3	Trailblazer	2008-08-03	False
4	RatSat	2008-09-28	True
5	RazakSat	2009-07-13	True
6	Falcon 9 Test Flight	2010-06-04	True

Archived response: 107 launch records. These rows show mission success, which differs from core landing_success.

Data collection: web scraping

1. Retrieve the Wikipedia launch-history HTML for 2010-2019.

2. Parse HTML tables with pandas.read_html and select tables containing payload and launch/date columns.

3. Flatten table headers and export seven selected tables as CSV.

4. Retain source HTML. Tables may contain Falcon Heavy, merged rows and footnotes, so they stay separate from modeling.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/02_Web_Scraping.ipynb

Data wrangling and target definition

Validate unique FlightNumber values and parse dates for yearly summaries.

Class 0: False ASDS, False Ocean, False RTLS, None ASDS or None None. Class 1: other recorded outcomes.

Confirm all 90 recreated labels match IBM dataset_part_2.csv.

Class 0 includes absent/unknown landing outcomes. This label is broader than physical landing failure.

The enriched CSV already includes upstream payload imputation. Model-specific preprocessing fits within training folds.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/03_Data_Wrangling.ipynb

EDA with visualization

Scatter plots: flight number and payload mass against launch site and orbit.

Bar charts: Class 1 proportion by site and orbit, with group sample sizes.

Line chart: yearly Class 1 proportion from the 90-row modeling snapshot.

Descriptive associations identify patterns. Small groups and changing launch practices limit causal interpretation.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/04_EDA_Visualization.ipynb

EDA with SQL

Load the cleaned 90-row snapshot into SQLite as launches.

Query distinct sites, site prefixes, payload totals/averages and outcome counts.

Rank site success proportions and retrieve maximum-payload flights.

Query first successful RTLS, selected ASDS payload ranges, yearly trends and outcome counts within a date window.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/05_EDA_SQL.ipynb / results/queries.sql

Interactive visual analytics

Folium: site circles, launch-record clusters, class colors, measurement controls and a proximity line.

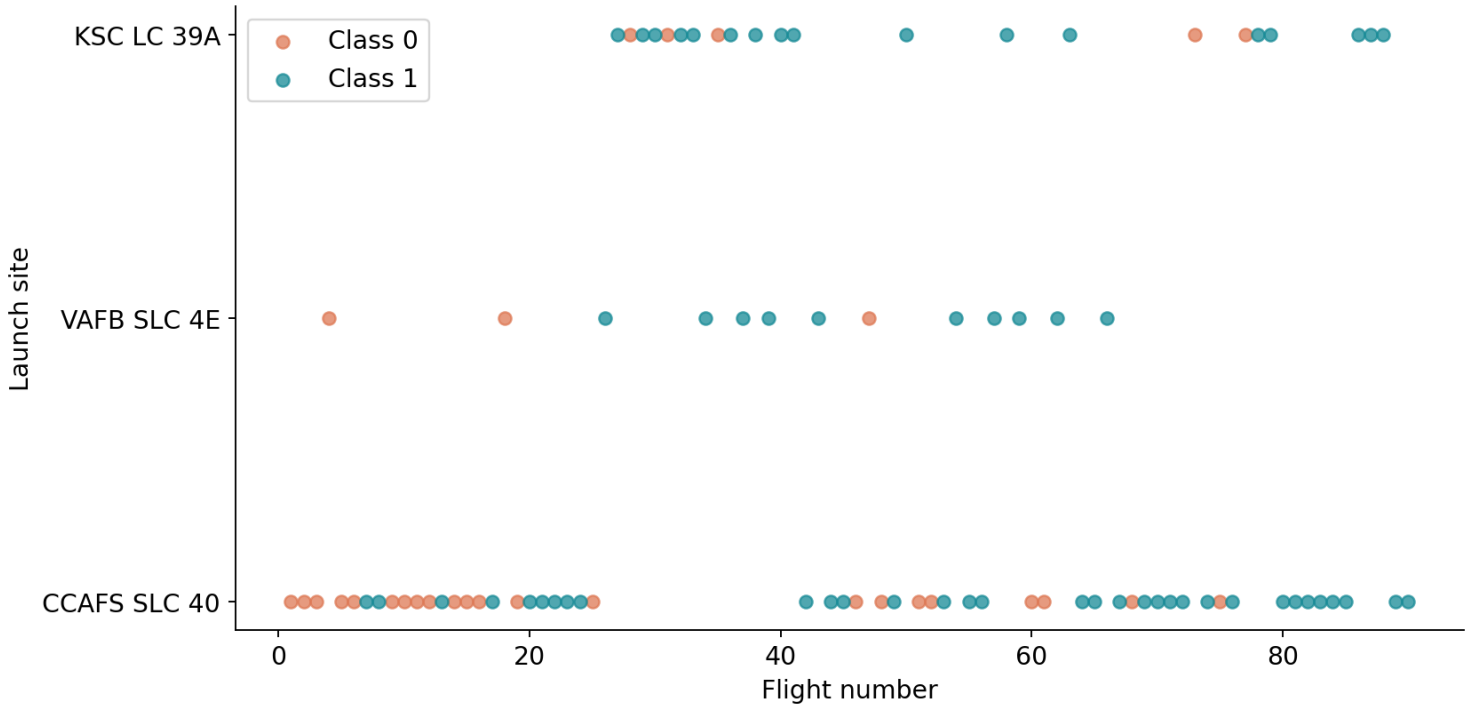
Dash: site dropdown, outcome pie charts and payload range slider controlling the scatter.

The all-sites pie reports shares of successful launch counts. The selected-site pie reports class proportions.

These tools use the 56-record interactive snapshots, independent of the modeling sample.

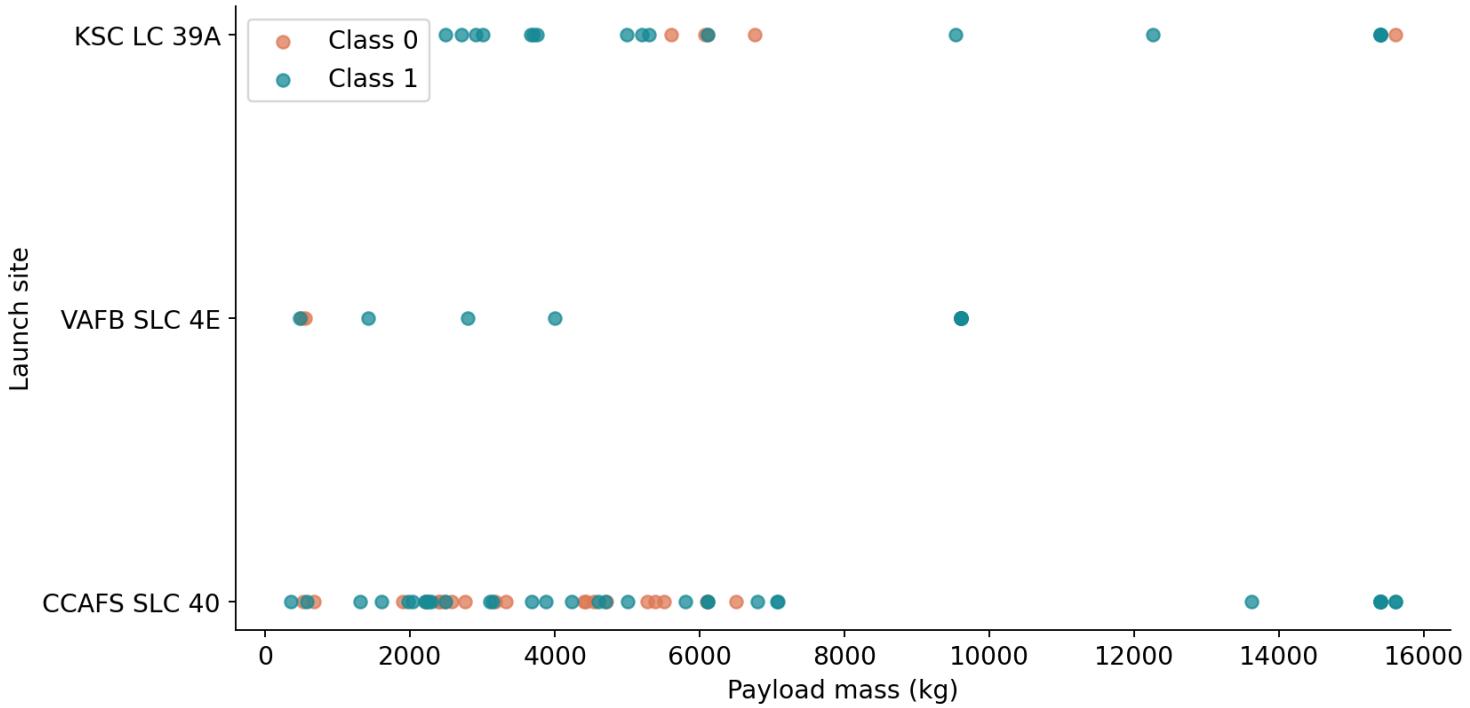
Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/06_Folium_Map.ipynb / notebooks/07_Plotly_Dash.ipynb / spacex_dash_app.py

Flight number and launch site



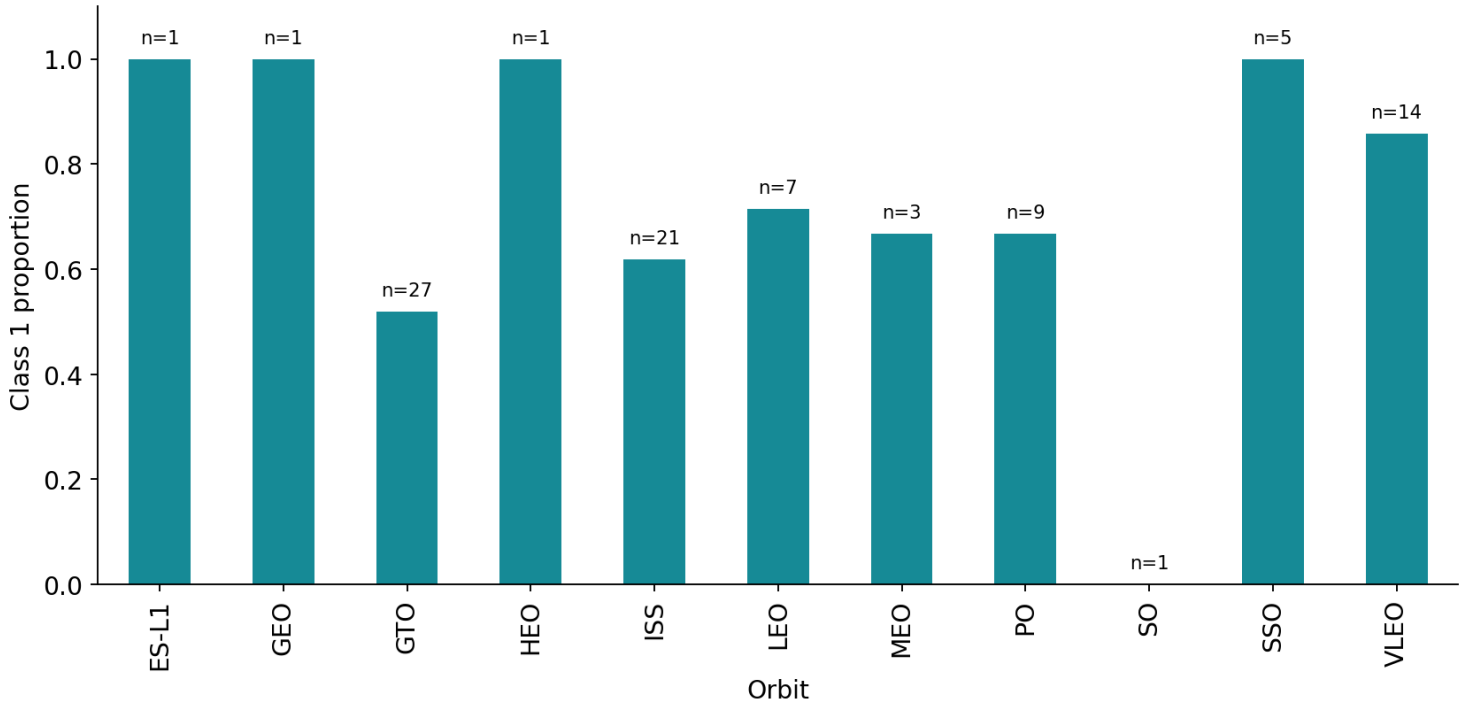
Later flights contain more Class 1 records. Flight number also tracks time and changes in launch configuration.

Payload mass and launch site



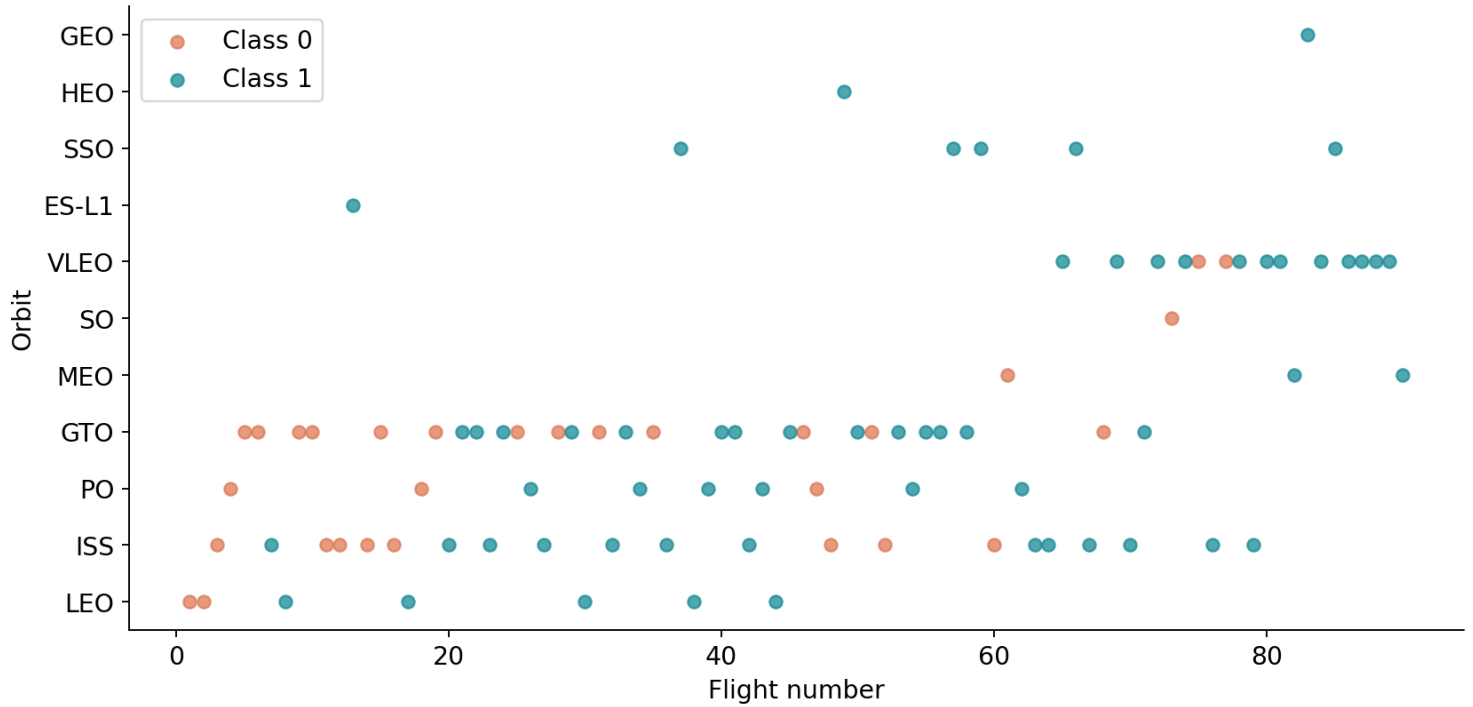
Successful and non-successful outcomes overlap across payload values. This plot does not establish a universal payload cutoff.

Class 1 proportion by orbit



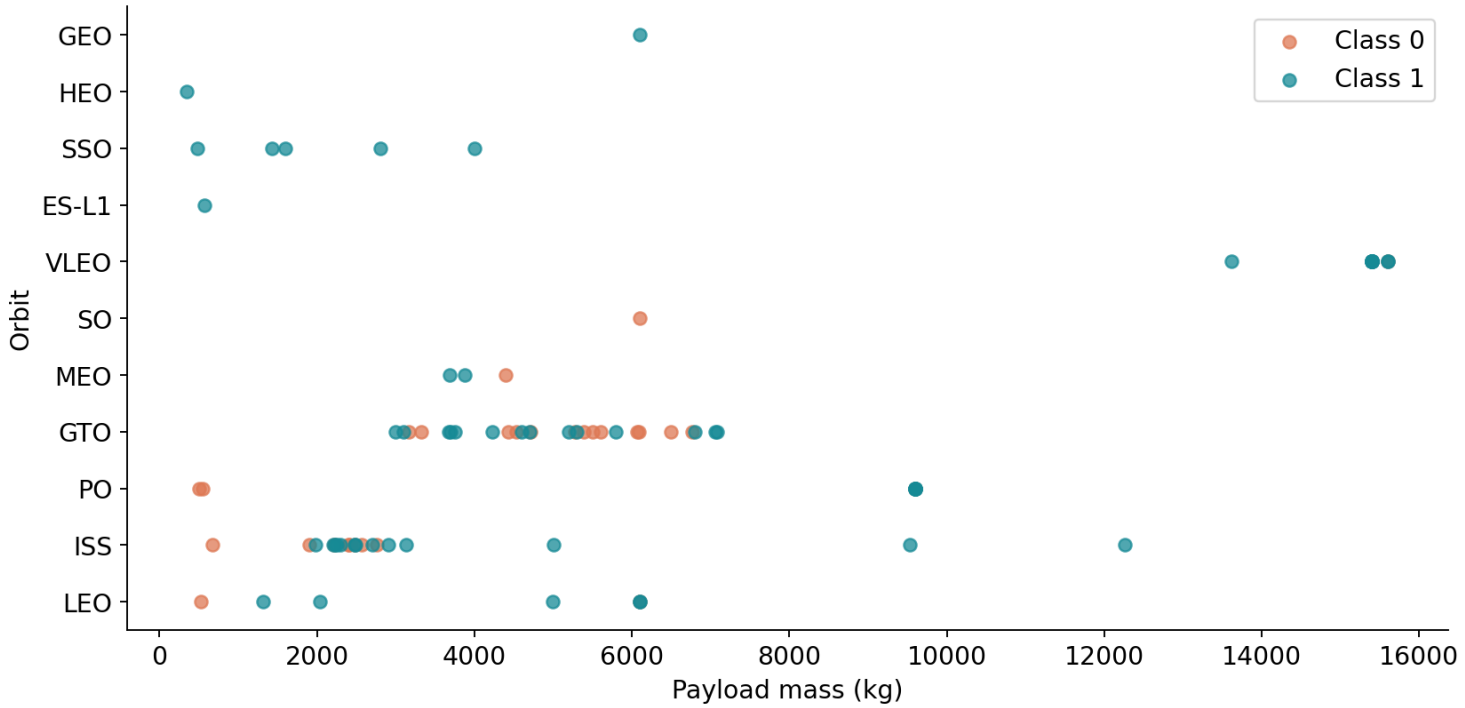
Group sample sizes appear above the bars. A high rate from a small orbit group is weak evidence of future performance.

Flight number and orbit



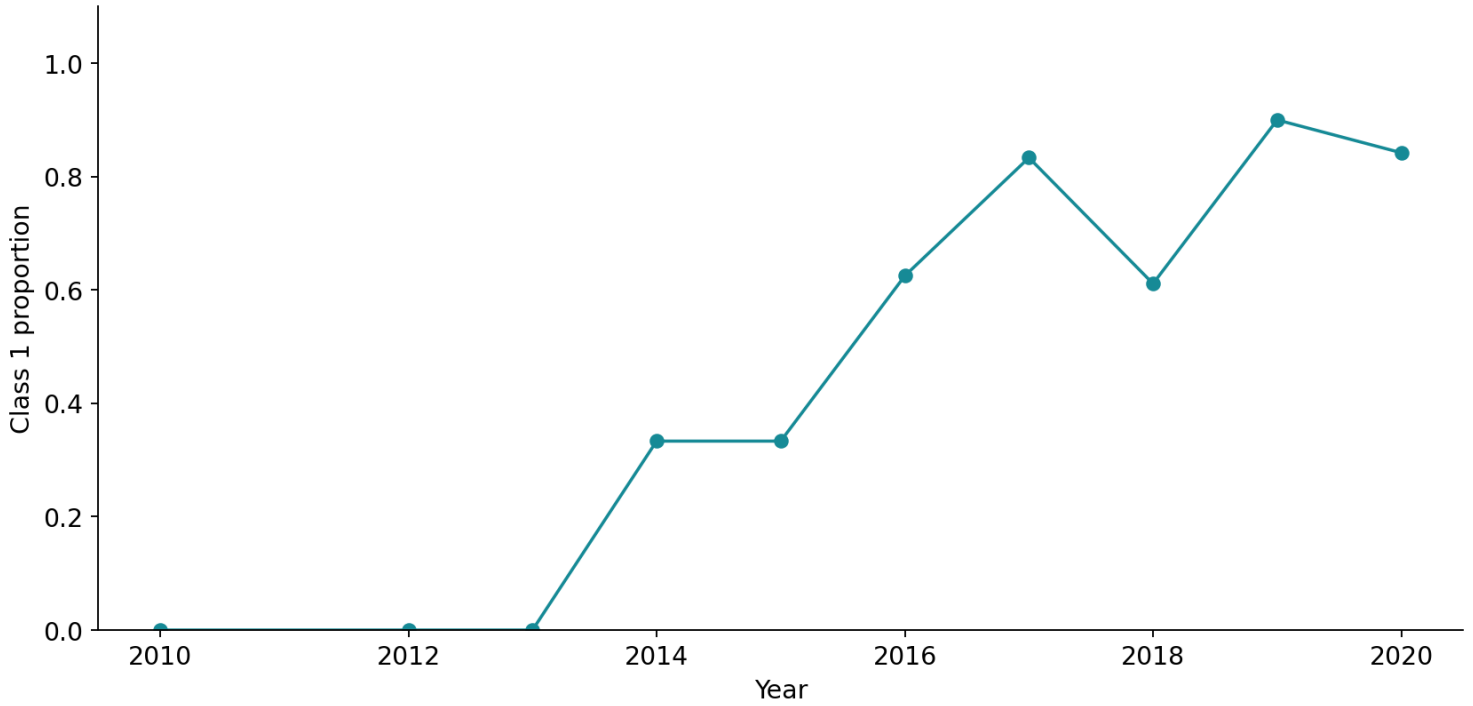
Flight sequence and orbital missions vary together. Orbit differences may reflect time and mission selection.

Payload mass and orbit



Payload ranges differ across orbits. Descriptive comparisons should account for orbit and mission requirements.

Yearly Class 1 proportion



Historical proportions generally improve in later years, with year-to-year variation. Future launches require a chronological evaluation.

SQL results: launch sites and payloads

LaunchSite	launches	total_kg	mean_kg
CCAFS SLC 40	55	305,151.428	5,548.208
KSC LC 39A	22	167,341.919	7,606.451
VAFB SLC 4E	13	76,953.000	5,919.462

Payload totals and means use the enriched IBM snapshot, including its upstream payload imputation.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: results/sql_payloads.csv / results/sql_launch_sites.csv

SQL results: site ranking

LaunchSite	launches	class1	class1_rate
KSC LC 39A	22	17	0.773
VAFB SLC 4E	13	10	0.769
CCAFS SLC 40	55	33	0.600

KSC LC 39A: 17/22 Class 1. VAFB SLC 4E: 10/13. CCAFS SLC 40: 33/55. Different mission mixes and sample sizes prevent a causal site ranking.

SQL results: outcomes and time

First successful RTLS in this snapshot: 2015-12-22.

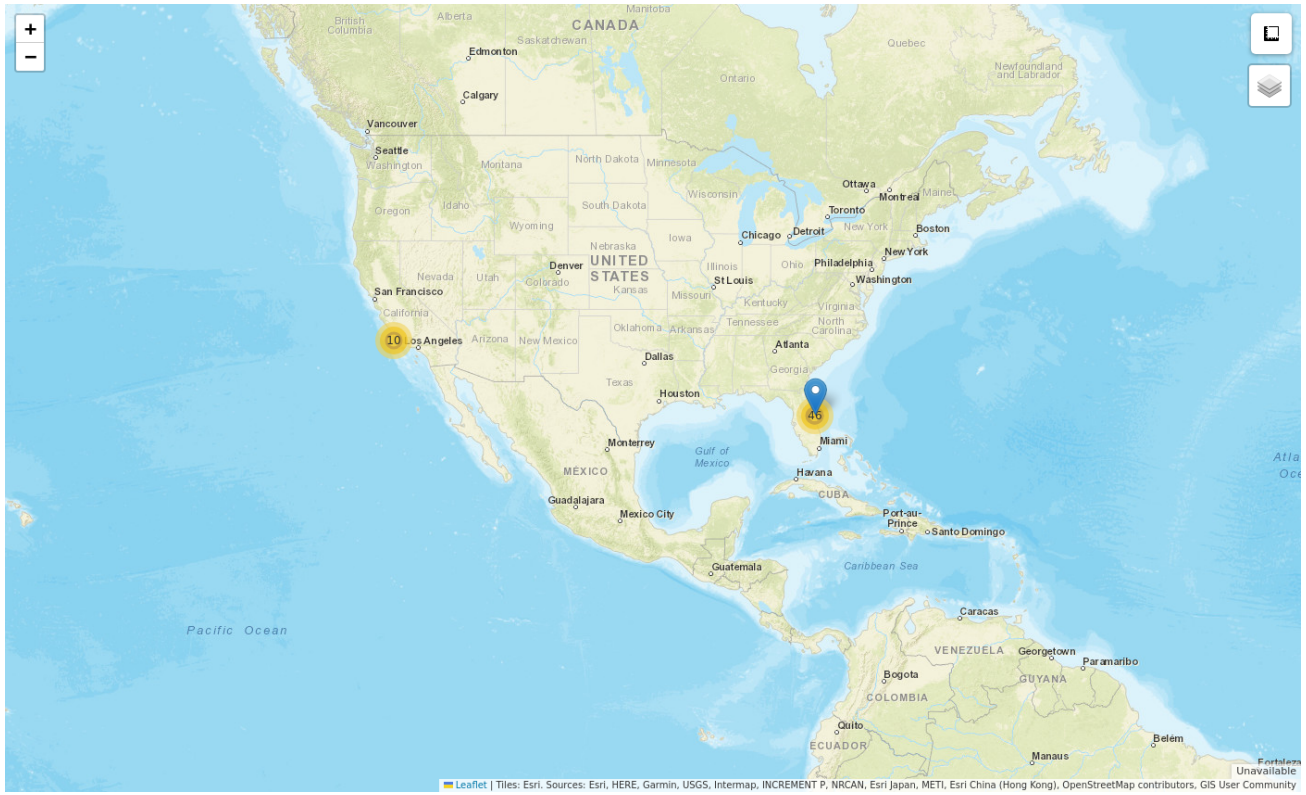
Maximum payload: 15,600 kg, flight 69, booster B1048.

Yearly counts, 2015 ASDS failures and date-window outcome rankings appear in the executed SQL notebook.

Full CSV outputs preserve all query results. Customer-specific SQL requires a customer field absent from this modeling snapshot.

Repository URL pending: upload project to GitHub and set repository_url.txt
Code: results/sql_*.csv / notebooks/05_EDA_SQL.ipynb

Folium launch sites and records



CCAFS LC-40

CCAFS SLC-40

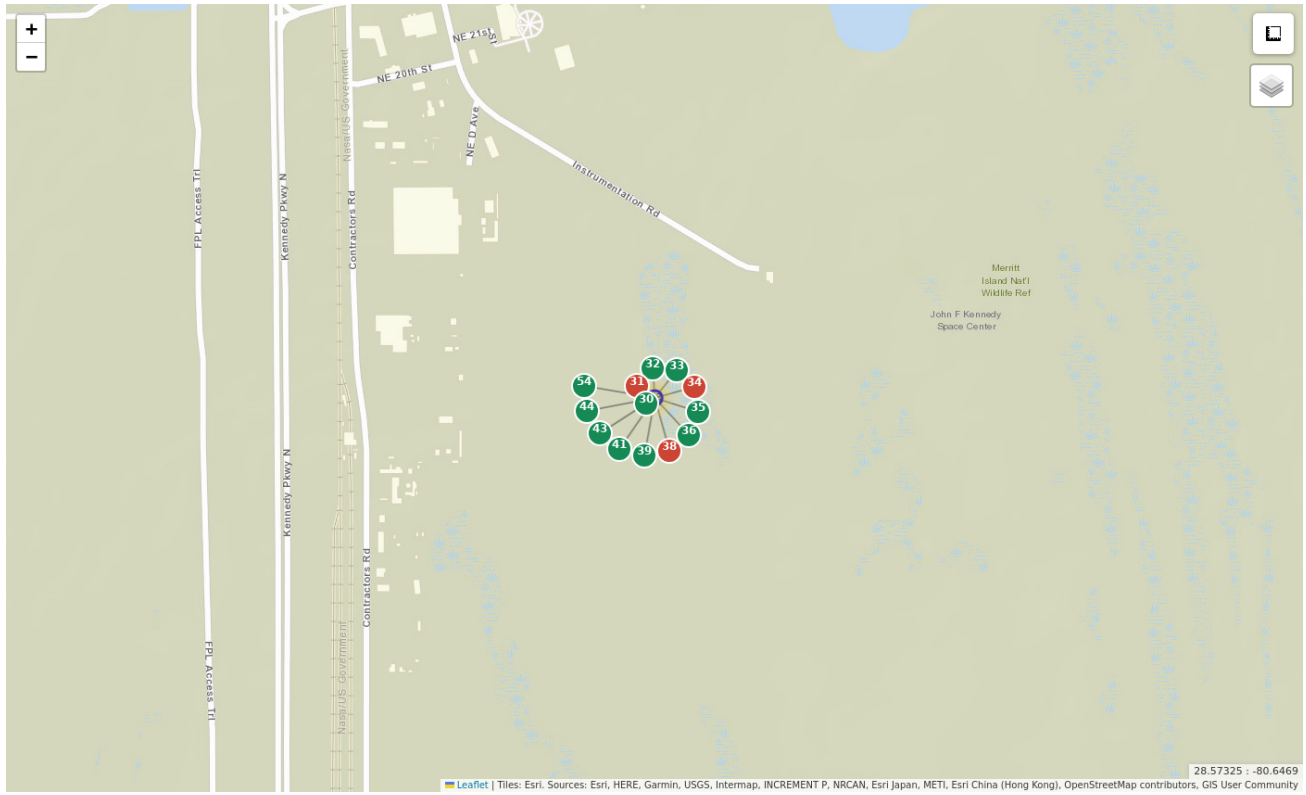
KSC LC-39A

VAFB SLC-4E

Cluster counts preserve overlapping records.

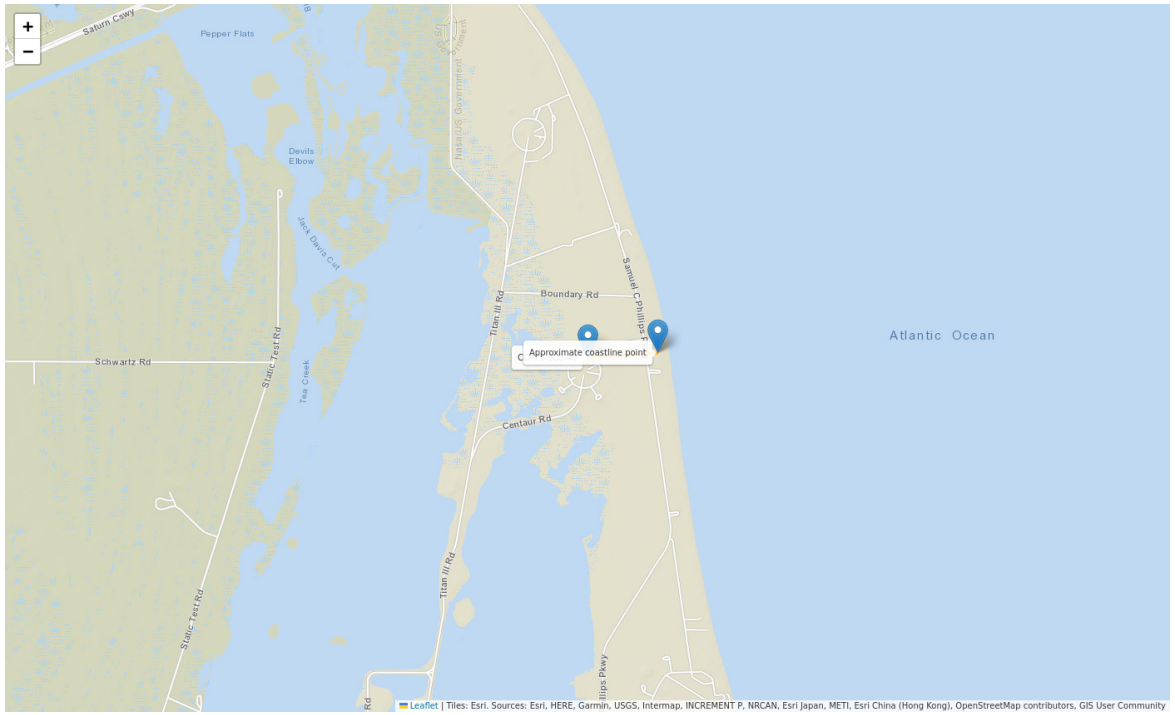
Four site labels in the 56-record geography snapshot. Marker clusters preserve overlapping launch records. Green marks Class 1 and red marks Class 0.

Folium individual launch records



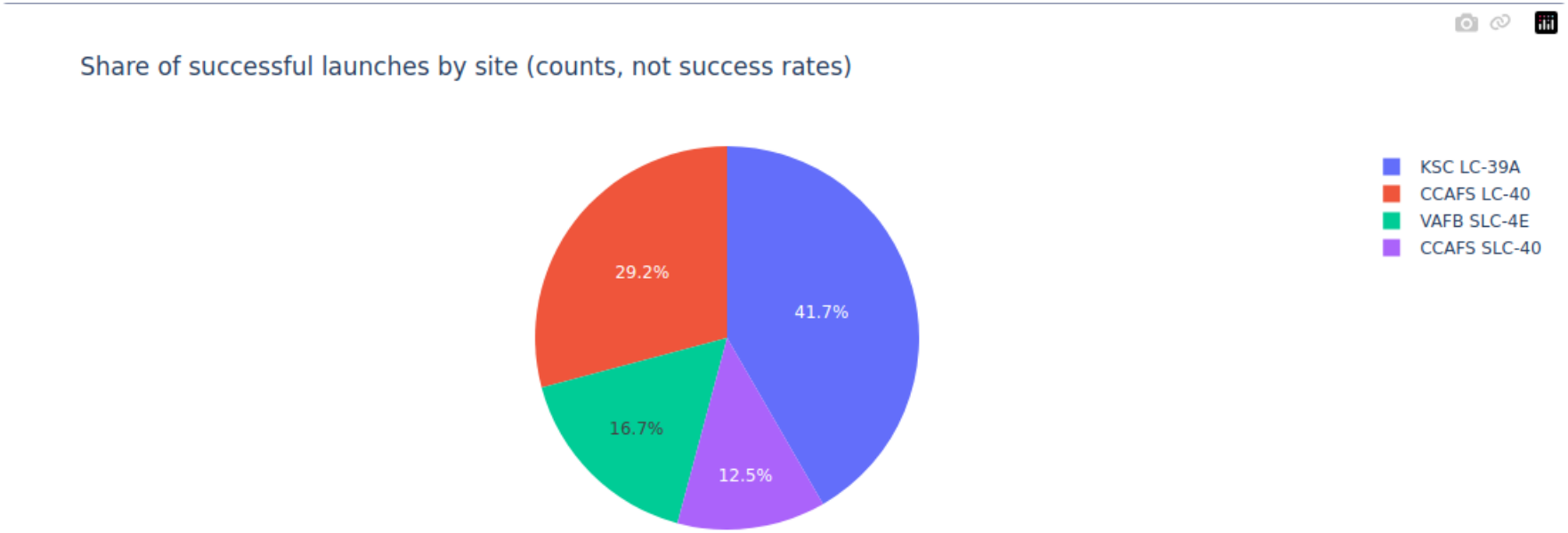
KSC LC-39A cluster expanded into individual launch records. Green markers denote Class 1 and red markers denote Class 0. Tooltips and popups identify each record.

Folium proximity analysis



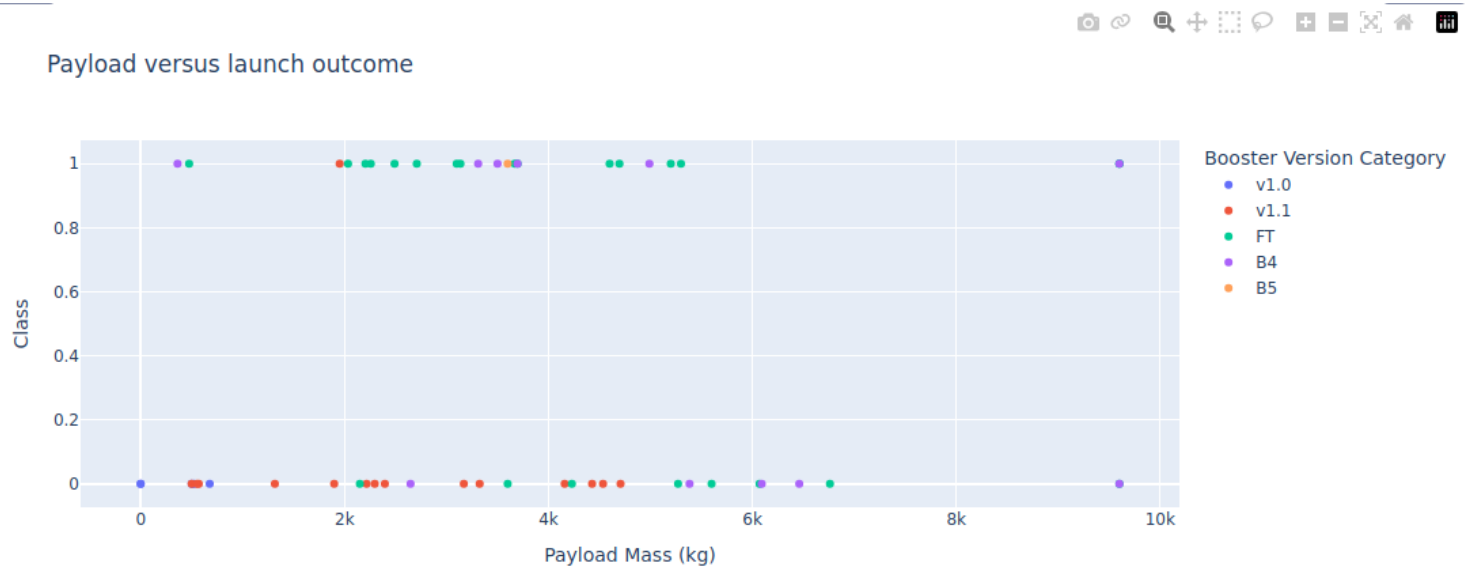
CCAFS SLC-40 to the selected approximate coastline point: 0.71 km using Haversine distance. This is a point-to-point illustration, not the shortest distance to the coastline.

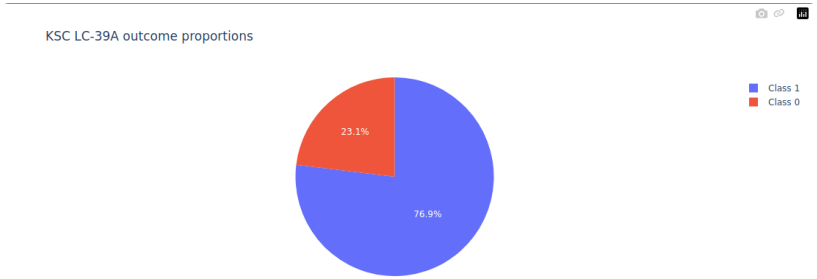
Dash: successful launch counts by site



All-sites view: share of successful launch counts in the 56-row dashboard dataset. A larger pie slice does not necessarily mean a higher success rate.

Dash: site filter and payload scatter





The dropdown changes the site view. The payload slider filters scatter points. Booster-version categories distinguish groups; their associations do not prove a causal effect.

Predictive analysis protocol

Stratified 80/20 split: 72 training records and 18 test records, seed 42.

Fivefold stratified CV on training data selects hyperparameters and the final model.

Numerical median imputation and scaling fit inside each CV fold. Categorical values use one-hot encoding.

Predictors: PayloadMass, Flights, Block, Orbit, LaunchSite, GridFins, Reused and Legs.

Exclude outcome/landing-pad information and fields with possible later reuse history. Inspect feature availability before deployment.

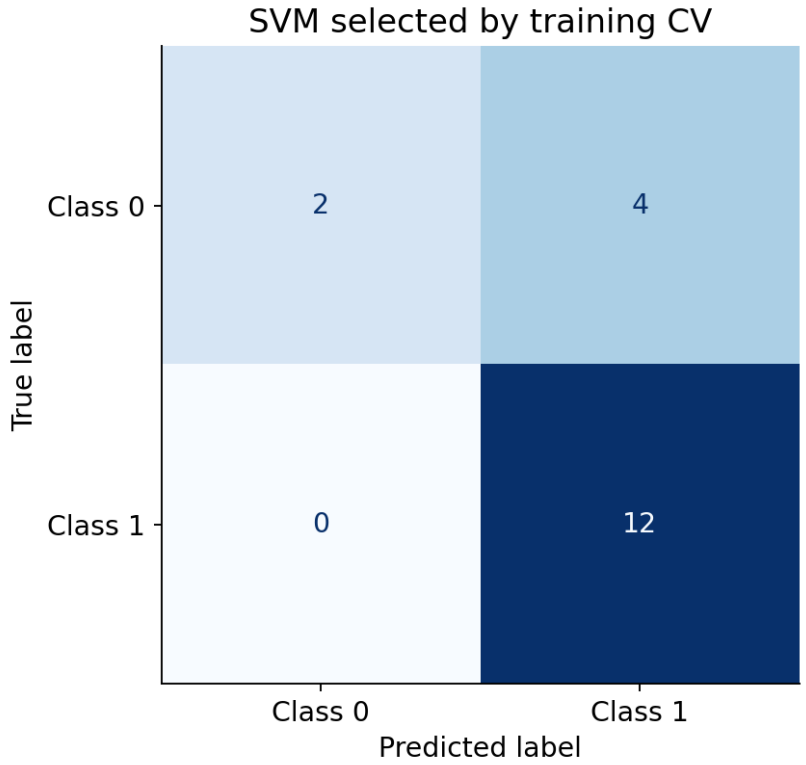
Repository URL pending: upload project to GitHub and set repository_url.txt
Code: notebooks/08_Predictive_Analysis.ipynb

Model comparison

Model	CV accuracy	Test accuracy	Test F1
SVM	0.889	0.778	0.857
Decision Tree	0.874	0.778	0.833
KNN	0.862	0.778	0.857
Logistic Regression	0.849	0.778	0.857

SVM leads training CV and uses an RBF kernel with C=1 and gamma=scale. All four test accuracies tie at 14/18. This small test does not support a definitive model ranking.

SVM confusion matrix



True Class 0: 2 correct, 4 predicted Class 1.

True Class 1: 12 correct, 0 predicted Class 0.

Accuracy: 77.78%. Class 1 F1: 85.71%.

Class 0 recall: 33.33%. Performance differs strongly by class.

Conclusion and next steps

The workflow links source collection, label validation, exploration and a reproducible four-model benchmark.

Site, orbit and time patterns provide descriptive context. Payload alone does not separate the outcomes.

The selected SVM performs better than the 66.67% majority baseline on this test, but misses four Class 0 cases.

Next: verify pre-launch feature availability, expand the data and evaluate a chronological holdout. Audit subgroup errors before operational use.

Repository and source references

GitHub URL pending. Create a public repository and add its URL to repository_url.txt.

Run python build_report.py to regenerate this PDF with the URL on the required methodology slides.

IBM source files, their SHA256 checksums and URLs: results/data_provenance.json.

Archived API source: IBM datasets/API_call_spacex_api.json. Wikipedia source HTML and scraped CSVs are included.

Full code: eight executed notebooks, pipeline.py and spacex_dash_app.py.

Wikipedia: List of Falcon 9 and Falcon Heavy launches (2010-2019)
API archive and CSVs: cf-courses-data / IBM-DS0321EN-SkillsNetwork/datasets